

## ■ Lab 5 — Outputs & Variables

### ■ Objectifs du Lab

À la fin de ce lab, vous aurez :

- Organisé votre code Terraform selon les **bonnes pratiques** de structuration des fichiers
- Exporté des attributs de ressources avec les **outputs**
- Déclaré et utilisé des **variables** avec différents types
- Assigné des variables via **CLI**, **tfvars** et **variables d'environnement**
- Ajouté des **règles de validation** sur les variables

## ■ Étape 1 — Structure des fichiers (bonnes pratiques)

```
cd labs/lab5-outputs
```

Un projet Terraform bien organisé sépare les responsabilités en fichiers distincts :

|                      |                                        |
|----------------------|----------------------------------------|
| lab5-outputs/        |                                        |
| ■■■ backend.tf       | ← Configuration du backend S3          |
| ■■■ versions.tf      | ← Version Terraform + providers requis |
| ■■■ provider.tf      | ← Configuration du provider AWS        |
| ■■■ variables.tf     | ← Déclaration des variables            |
| ■■■ main.tf          | ← Ressources uniquement                |
| ■■■ outputs.tf       | ← Outputs                              |
| ■■■ terraform.tfvars | ← Valeurs des variables                |

■ Cette séparation facilite la lecture, la maintenance et la réutilisation du code. C'est la convention adoptée par la majorité des équipes Terraform en production.

## ■ Étape 2 — backend.tf

```
`backend.tf`
```

```
terraform {  
  backend "s3" {  
    region      = "eu-west-1"  
    use_lockfile = true  
    encrypt     = true  
  }  
}
```

■ Le backend est isolé dans son propre fichier pour faciliter le changement de backend (local → S3 → Terraform Cloud) sans toucher aux autres fichiers.

### ■ Étape 3 — versions.tf

#### `versions.tf`

```
terraform {  
  required_version = ">= 1.15.0"  
  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "~> 5.0"  
    }  
  }  
}
```

### ■ Étape 4 — provider.tf

#### `provider.tf`

```
provider "aws" {  
  region = var.region  
}
```

■ Le provider est dans son propre fichier — si vous avez plusieurs providers (AWS + Azure par exemple), chacun aura son fichier `provider-aws.tf`, `provider-azure.tf`, etc.

### ■ Étape 5 — variables.tf

### `variables.tf`

```
variable "username" {
  description = "Votre prenom - permet de differencier les ressources entre participants"
  type        = string

  validation {
    condition     = length(var.username) >= 2 && length(var.username) <= 20
    error_message = "Le username doit contenir entre 2 et 20 caractères."
  }
}

variable "region" {
  description = "Region AWS de déploiement"
  type        = string
  default     = "eu-west-1"
}

variable "instance_type" {
  description = "Type d'instance EC2"
  type        = string
  default     = "t2.micro"

  validation {
    condition     = contains(["t2.micro", "t2.small", "t2.medium"], var.instance_type)
    error_message = "Le type d'instance doit être t2.micro, t2.small ou t2.medium."
  }
}

variable "environment" {
  description = "Environnement de déploiement"
  type        = string

  default     = "dev"

  validation {
    condition     = contains(["dev", "prod"], var.environment)
    error_message = "L'environnement doit être dev ou prod."
  }
}
```

## ■ Étape 6 — main.tf

### `main.tf`

```
resource "aws_security_group" "lab5_sg" {
  name          = "lab5-sg-${var.username}-${var.environment}"
  description    = "Security group Lab 5 - ${var.username}"

  tags = {
    Name      = "lab5-sg-${var.username}"
    Lab       = "lab5"
    Username  = var.username
    Environment = var.environment
  }
}

resource "aws_instance" "lab5_instance" {
  ami              = "ami-0694d931cee176e7d"
  instance_type    = var.instance_type
  vpc_security_group_ids = [aws_security_group.lab5_sg.id]

  tags = {
    Name      = "lab5-instance-${var.username}"
    Lab       = "lab5"
    Username  = var.username
    Environment = var.environment
  }
}
```

■ `main.tf` contient **uniquement les ressources**. Plus le projet grandit, plus cette séparation est importante.

## ■ Étape 7 — outputs.tf

`outputs.tf`

```
output "instance_id" {
  description = "ID de l'instance EC2"
  value       = aws_instance.lab5_instance.id
}

output "instance_public_ip" {
  description = "IP publique de l'instance"
  value       = aws_instance.lab5_instance.public_ip
}

output "security_group_id" {
  description = "ID du Security Group"
  value       = aws_security_group.lab5_sg.id
}

output "security_group_name" {
  description = "Nom du Security Group"
  value       = aws_security_group.lab5_sg.name
}

output "deployment_summary" {
  description = "Résumé du déploiement"
  value = {
    username    = var.username
    environment = var.environment
    instance    = aws_instance.lab5_instance.id
    region      = var.region
  }
}
```

■ Les outputs peuvent exposer :

- Un **attribut spécifique** : ``aws_instance.lab5_instance.id``

- Un **objet complet** : comme ``deployment_summary``

- Dans un module, les outputs sont accessibles par d'autres ressources

## ■ Étape 8 — Initialiser avec le Backend S3

```
BUCKET_NAME=$(cat $HOME/tf-training-info.txt | awk '{print $NF}')
```

```
terraform init \
  -backend-config="bucket=${BUCKET_NAME}" \
  -backend-config="key=terraform.tfstate"
```

## ■ Étape 9 — Les 3 façons d'assigner les variables

### Méthode 1 — CLI flag (`-var`)

```
terraform apply \  
-var="username=<votre-prenom>" \  
-var="environment=dev" \  
-var="instance_type=t2.micro"
```

### Méthode 2 — Fichier tfvars

Créez `terraform.tfvars` :

```
username      = "<votre-prenom>"  
environment    = "dev"  
instance_type = "t2.micro"
```

Puis appliquez sans `-var` :

```
terraform apply
```

■ Terraform charge automatiquement `terraform.tfvars` s'il est présent dans le dossier. Pour un fichier différent :  
`terraform apply -var-file="prod.tfvars"`

### Méthode 3 — Variables d'environnement (`TF\_VAR\_`)

```
export TF_VAR_username="<votre-prenom>"  
export TF_VAR_environment="dev"  
export TF_VAR_instance_type="t2.micro"  
  
terraform apply
```

■ Le préfixe `TF_VAR_` suivi du nom exact de la variable est reconnu automatiquement par Terraform.

## ■ Étape 10 — Tester la validation

```
# Tester une valeur d'environnement invalide  
terraform plan -var="username=<votre-prenom>" -var="environment=staging"
```

Résultat attendu :

Error: Invalid value for variable  
L'environnement doit être dev ou prod.

```
# Tester un type d'instance invalide
terraform plan -var="username=<votre-prenom>" -var="instance_type=t3.large"
```

Résultat attendu :

Error: Invalid value for variable  
Le type d'instance doit être t2.micro, t2.small ou t2.medium.

## ■ Étape 11 — Utiliser terraform output

Après le `terraform apply` :

```
# Afficher tous les outputs
terraform output

# Afficher un output spécifique
terraform output instance_id

# Afficher en JSON (utile pour les scripts)
terraform output -json deployment_summary

# Afficher brut sans guillemets (utile dans les pipelines CI/CD)
terraform output -raw instance_id
```

## ■ Étape 12 — Nettoyage

```
terraform destroy -var="username=<votre-prenom>"
```

## ■ Validation du Lab

| # | Critère                                                          | Validé |
|---|------------------------------------------------------------------|--------|
| 1 | Fichiers séparés : `backend.tf`,<br>`versions.tf`, `provider.tf` | ■      |

| # | Critère                                                           | Validé |
|---|-------------------------------------------------------------------|--------|
| 2 | `terraform init -backend-config` réussi                           | ■      |
| 3 | Validation bloque `environment=staging`                           | ■      |
| 4 | Validation bloque `instance_type=t3.large`                        | ■      |
| 5 | `terraform apply` via `-var` fonctionne                           | ■      |
| 6 | `terraform apply` via `terraform.tfvars` fonctionne               | ■      |
| 7 | `terraform apply` via `TF_VAR_` fonctionne                        | ■      |
| 8 | `terraform output -json deployment_summary` affiche un objet JSON | ■      |
| 9 | `terraform destroy` supprime les ressources                       | ■      |

## ■ ■ Résolution des problèmes courants

| Problème                                | Cause                                                               | Solution                                              |
|-----------------------------------------|---------------------------------------------------------------------|-------------------------------------------------------|
| `Error: No value for required variable` | Variable sans default non fournie                                   | Ajouter `-var="username=..."` ou `terraform.tfvars`   |
| `Error: Invalid value for variable`     | Validation échouée                                                  | Vérifier la valeur fournie                            |
| `TF_VAR_` ignoré                        | Mauvais nom de variable                                             | Vérifier l'orthographe exacte                         |
| `Duplicate terraform block`             | `backend.tf` et `versions.tf` ont tous les deux un bloc `terraform` | Normal — Terraform fusionne les blocs automatiquement |

■ **\*\*Fin du Lab 5 — Votre code est bien structuré et dynamique !\*\***

*Le Lab 6 introduit les **\*\*locals\*\*** pour simplifier et centraliser les expressions dans votre code.*